Matteo Liotta, matteo.liotta@studenti.units.it

Esercitazione 9: *Macchine di Turing (cont'd)*

Nota. Sono indicati in *blu* gli esercizi che non saranno trattati durante l'esercitazione, bensì lasciati come strumento aggiuntivo di esercitazione individuale. Resta comunque assoluta disponibilità per la loro risoluzione negli incontri successivi, qualora necessario.

Esercizio 52 (*)
Sia L linguaggio su $\Sigma = \{a, b\}$ (cfr. *approfondimento 02*), tale per cui:

$$L = \{w\#w' | w, w' \in \Sigma^*, \quad \exists i \in \mathbf{N} \quad t.c. \quad w(i) = w'(i)\ \}$$

Si rappresenti la macchina di Turing tale da riconoscere il linguaggio L.

Esercizio 53. (*)
Sia L linguaggio su $\Sigma = \{a, b, c\}$, tale per cui

$$L = \{x\#y \mid x, y \in \Sigma^* \quad e \quad x \text{ prefisso di } y\}$$

Si rappresenti la macchina di Turing tale da riconoscere il linguaggio L.

Esercizio 54. (*)
Sia L linguaggio su $\Sigma = \{a, b, c\}$, tale per cui

$$L = \{x\#y \mid x, y \in \Sigma^* \quad e \quad x \text{ sottostringa di } y\}$$

Si rappresenti la macchina di Turing tale da riconoscere il linguaggio L.

Esercizio 55. (***)
Si considerino le seguenti identificazioni, considerando il seguente alfabeto $\{0, 1\}$ e simboli ausiliari $\{\$_1, \$_2, \$_c, \#_1, \#_2, \sqcup\}$:

- `00` $\equiv$ `IF` | `01` $\equiv$ `ELSE`
- `0` $\equiv \leq$ | `1` $\equiv >$
- `10` $\equiv$ `AND` | `1` $\equiv$ `OR (element-wise)`

Dove sono presenti le seguenti regole sintattiche:

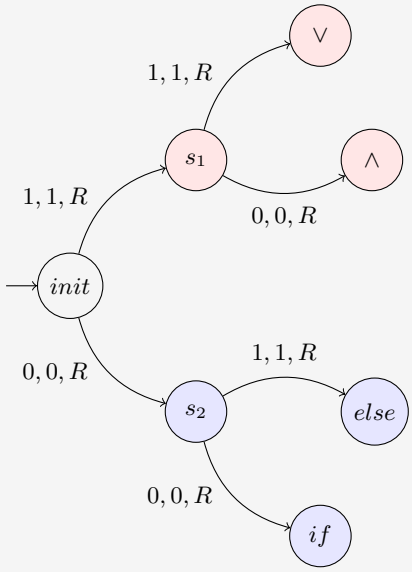
- Ogni operazione di controllo con if richiede l'indicazione del termine di paragone voluto, i termini da confrontare, correttamente separati:
  `IF [binary num 1] [binary num 2] CONFRONTO THEN [what to do]` $\equiv$ `IF`$\$_1 x \$_2 y \$_c c \#_1 command \#_2$
  $x, y \in \Sigma^*$
- Si definisce l'operazione alternativa, posta dopo la codifica del THEN dell'IF:
  `ELSE [what to do]` $\equiv$ `ELSE`$\#_1 command \#_2$
- Mentre, con operazioni (then), in questo caso si sono considerate somme e sottrazioni elementwise degi termini:
  `OPERATION [binary num 1] [binary num 2]` $\equiv$ `OP`$\$_1 x \$_2 y \#_2 \quad x, y \in \Sigma^*$

Si scriva una macchina di Turing che, ricevendo in input una stringa del programma (vedi esempio), ritorni sul nastro il risultato finale dell'(ultima) operazione

e.g. Se 010 > 001 allora effettuane l'AND:

- `INPUT:` $00\$_1 010\$_2 001\$_c 1\#_1 10\$_1 010\$_2 001\#_2$
- `OUTPUT: 011`

Soluzione.
Consideriamo dapprima che, all'inizio, possiamo sempre solo osservare quattro casistiche: `00,01,10,11`:

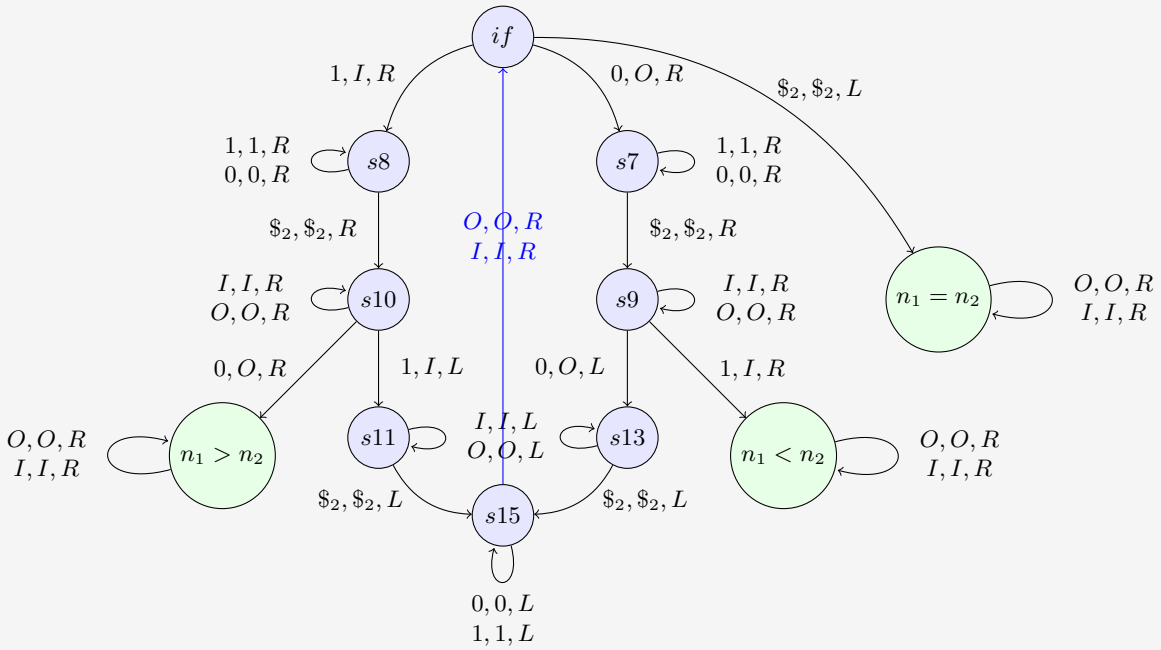


A questo punto, ricordiamo che nel caso di IF ci aspettiamo anche un confronto, che sarà indicato da 1/0. Sarebbe utile costruire una sottoporzione specializzata della macchina, tale da mappare in due nodi in cui viene mantenuta l'informazione implicita del rapporto tra i numeri. In questo modo, tramite la semplice lettura del termine di confronto successivo, potremo mappare in uno stato in cui è mantenuta la situazione di verità del confronto.

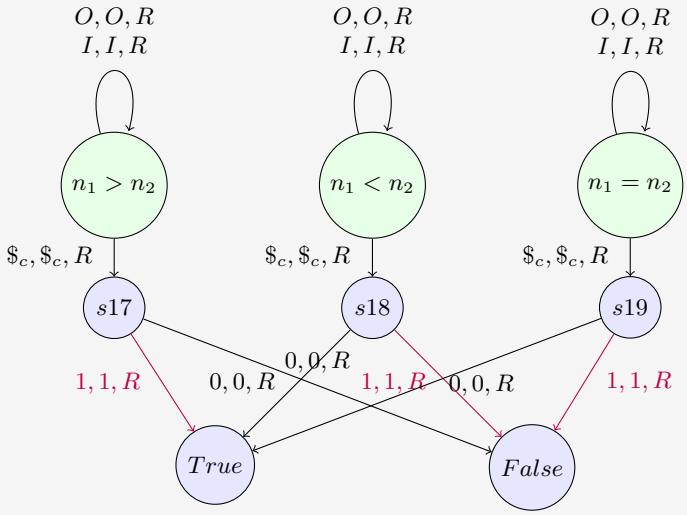
Per farlo, consideriamo dapprima il controllo binario element-wise: poiché stiamo considerando numeri binari, è sufficiente che la macchina controlli il *primo elemento per cui i numeri differiscono*; il primo dei due che avrà 1, sarà il maggiore.

Consideriamo la lettura del primo numero, in cui ricordiamo:

- Se $w_1(1:i-1) = w_2(1:i-1)$ e $w_1(i) = 1, w_2(i) = 0$ allora $w_1 > w_2$
- Se $w_1(1:i-1) = w_2(1:i-1)$ e $w_1(i) = 0, w_2(i) = 1$ allora $w_1 < w_2$
- Se $w_1(1:i-1) = w_2(1:i-1)$ e $w_1(i) = 0, w_2(i) = 0$ o $1 \quad \forall i$ allora $w_1 = w_2$



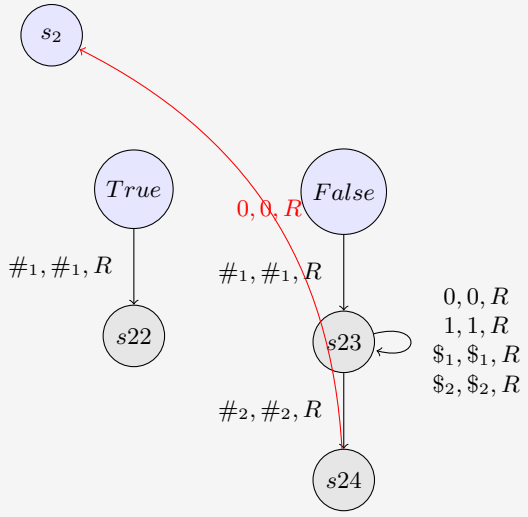
A questo punto sappiamo in che situazione si trovano, rispetto al confronto, i due addendi, quando gestiti dall'IF. Questo controllo viene effettuato sui termini, ma solo a seguito della lettura del termine di confronto, $>$ o $\leq$, potremo raggiungere uno stato in cui ci si riconduce alla condizione di verità propria della condizione dell'IF:



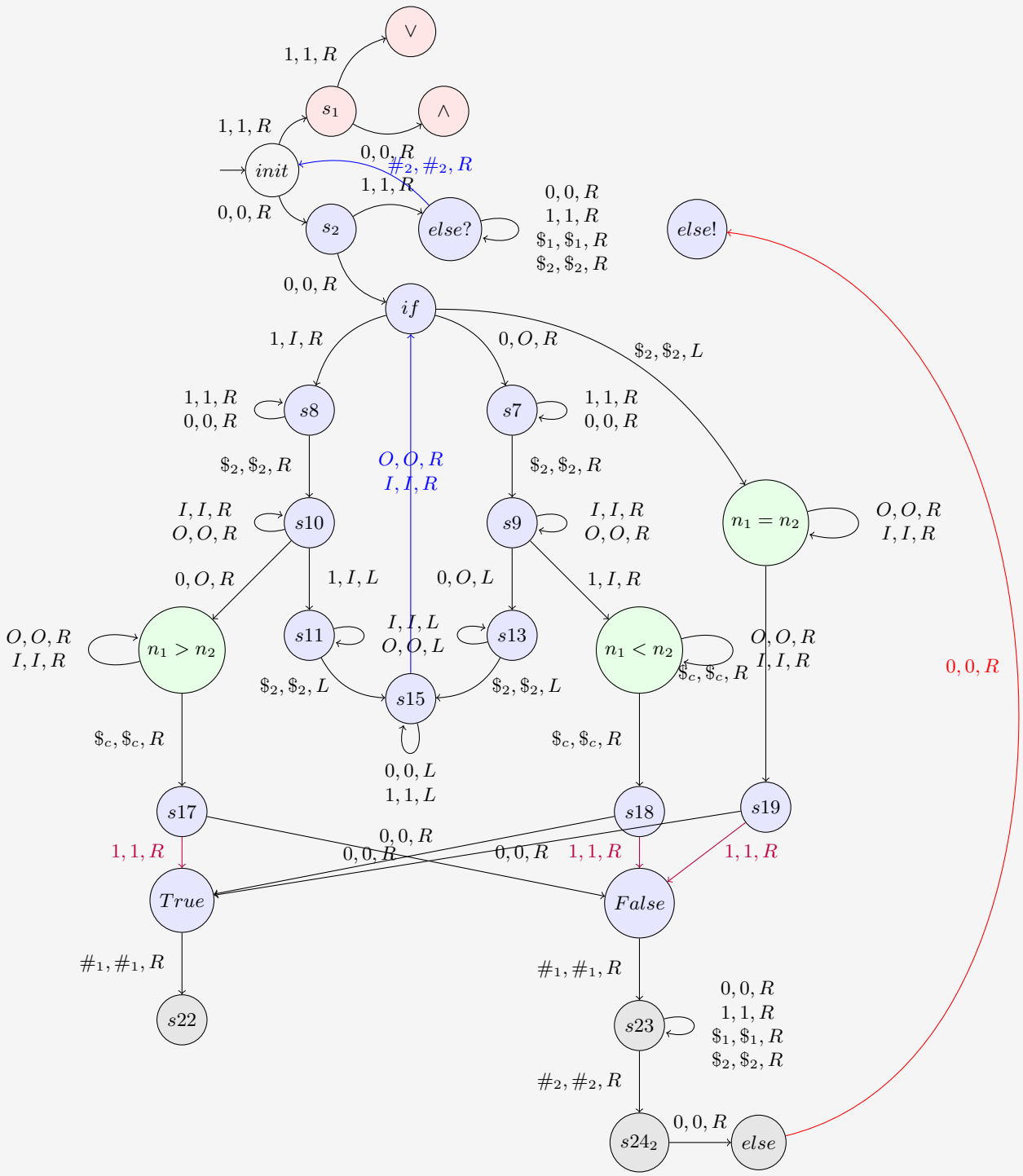
Così, se ad esempio da $n_1 > n_2$ osservo che il controllo era $>$ (codificato con 1), allora la condizione dell'IF è verificata. A questo punto, abbiamo una MDT che, ricevendo una codifica della forma $\texttt{00}\$_1\texttt{010}\$_2\texttt{001}\$_c\texttt{1}$ sul nastro, riesce a ritornare, riesce a ritornare la condizione di verità (ossia a mappare ad uno stato in cui è contenuta implicitamente quella condizione, true o false).
A questo punto mancano ancora la gestione della condizione THEN, la gestione dell'ELSE e la gestione delle operazioni element-wise. (Faremo riferimento al codice della precedente esercitazione per la costruzione di AND logico).
Si noti che:

- Ci aspettiamo l'esecuzione di un `ELSE` solamente dopo la falsità della condizione
- Se a True, la sezione *ELSE* dovrà essere ignorata. Ci aspettiamo che sia dopo il corpo dell'IF, come indicato in consegna. Si noti che, dopo aver eseguito una qualsiasi zona di operazioni, non ci aspettiamo di poter leggere 01 (else): sarà obbligatorio provenire da una condizione di verità. Dunque, se accade, salteremo tutti i termini fino a $\#_2$
- Quindi, se a False, *ignoreremo tutto il corpo dell'*`IF`

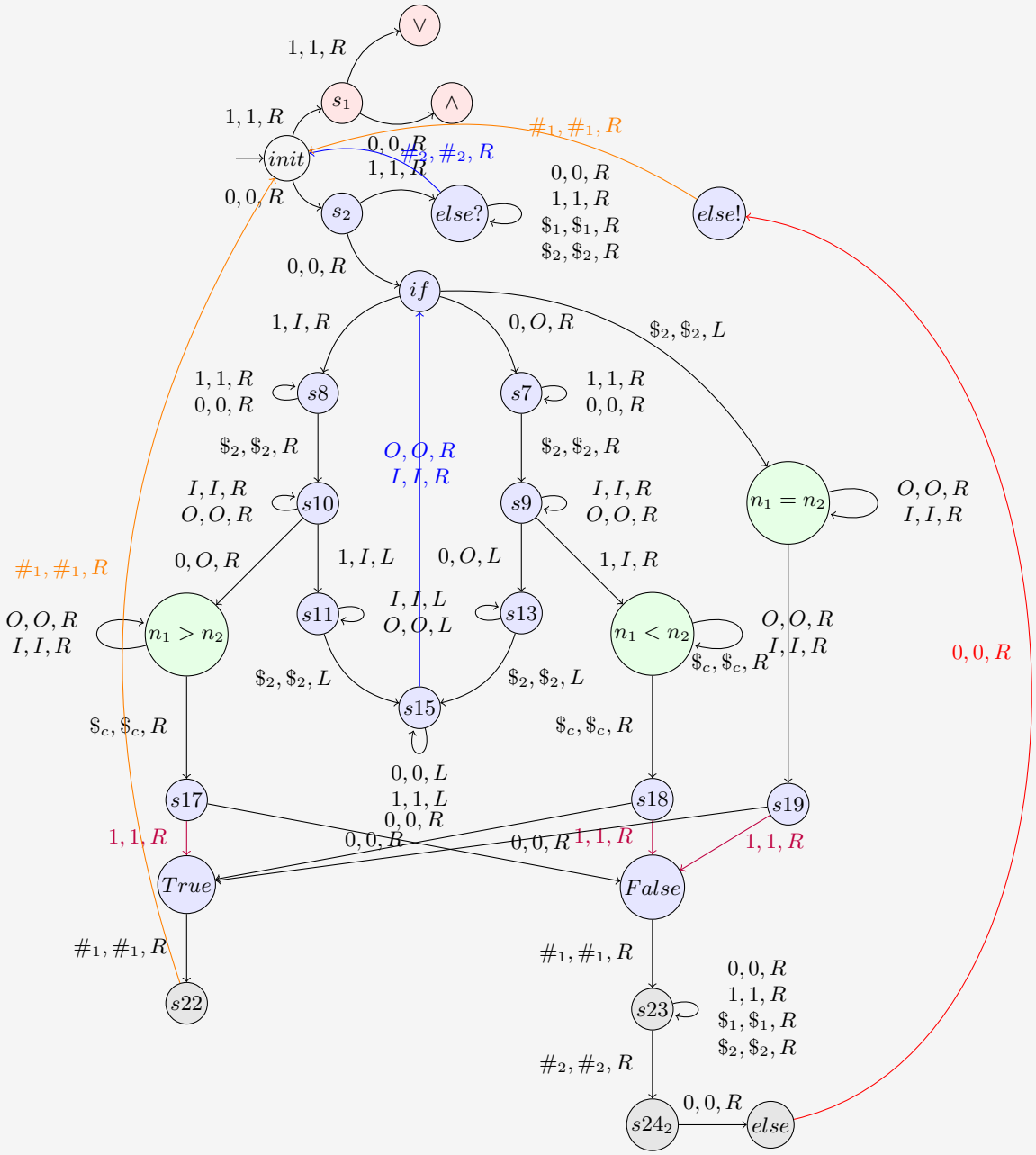
Arriviamo, poiché ora più semplice, alla situazione in cui siamo pronti, in ambo i casi `IF/ELSE` a gestire per iniziare a considerare il relativo `THEN`.



Si noti come la connesione rimanda, per semplicità al caso di biforcazione if/else. Ci aspettiamo comunque sia seguito da uno zero, ma si preferisce per evitare l'aggiunta di un nodo. Questa che segue è la condizione corrente della costruzione: possiamo dire ora che siamo pronti a gestire e introdurre le operazioni element-wise.



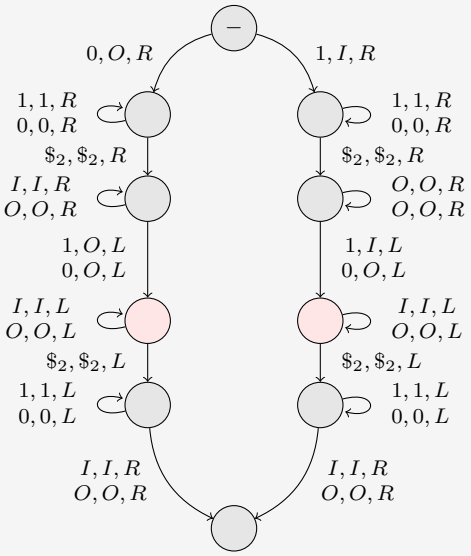
Dopo i controlli, possiamo **ricondurci al nodo iniziale**: il blocco di operazioni *then* può anche contenere altri controlli. Collegando anche questi stati, è sufficiente gestire le operazioni nei nodi indicati in rosso.



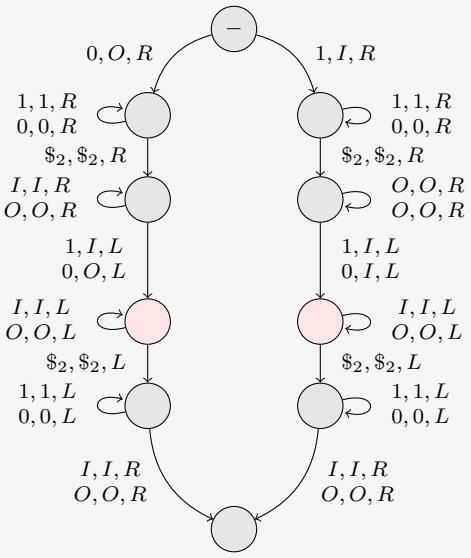
Dobbiamo considerare allora due aspetti:

- Per effettuare l'AND logico elemento per elemento, sarà sufficiente andare a controllare che ad ogni passo i-esimo ci sia un termine a 1 e 1 per scrivere 1
- Per effettuare l'OR logico elemento per elemento, sarà sufficiente andare a controllare che ad ogni passo i-esimo ci sia almeno un termine a 1 per scrivere 1.
- Scriveremo il risultato nella seconda parola: se questa è la fine del programma, cancelleremo tutto quello che la precede.

Utilizziamo dapprima il codice per l'AND della precedente esercitazione. **Si assume che in input siano presenti parole di lunghezza uguale**:



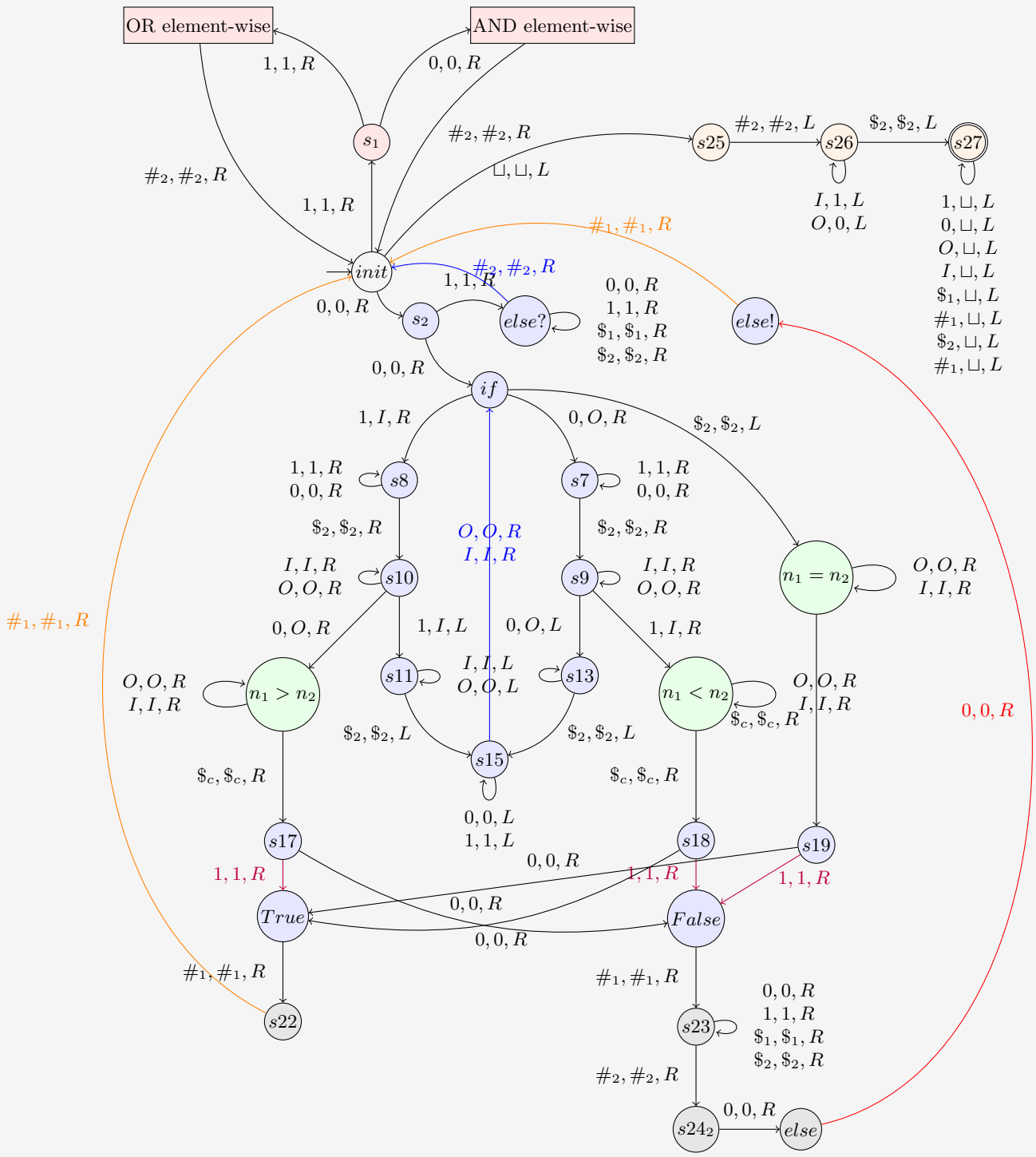
Possiamo allora generalizzare anche il caso OR a partire da questo:



A questo punto, la MdT lascia il termine risolto al posto della seconda parola. Poiché ci interessa solamente dell'esecuzione dell'**ultimo** (per semplicità), possiamo verificare se la codifica del programma è terminata: controlliamo che dopo $\#_2$ ci sia $\sqcup$. Per semplicità, li indicheremo con dei blocchi, che faranno riferimento a queste sottoporzioni.

Se così, vorremo tenere nel nastro solamente la parola:

- Cancelleremo qualsiasi cosa dal nastro, tranne quanto contenuto tra l'ultimo $\$_2 ... \#_2$
- Poi, sarà possibile riscrivere la parola all'inizio, ma lo eviteremo per semplicità.

Matteo Liotta, `matteo.liotta@studenti.units.it` 04/11/2025

MdT Finale: